In [ ]:
```
import numpy as np
from scipy.linalg import lu   # only used in Cell 5
```

In [ ]:
```
print("=" * 60)
print("TASK 1 — Array Creation & Manipulation")
print("=" * 60)

# --- creation ---
a = np.array([1, 2, 3, 4, 5])                       # 1-D array
A = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])    # 2-D array
z = np.zeros((3, 4))                                 # all zeros
o = np.ones((2, 5))                                  # all ones
e = np.eye(4)                                        # identity
r = np.arange(0, 20, 2)                              # evenly spaced values
l = np.linspace(0, 1, 6)                             # 6 points in [0,1]
R = np.random.rand(3, 3)                             # random in [0,1)

print("1-D array          :", a)
print("2-D array:\n",           A)
print("zeros(3,4):\n",          z)
print("ones(2,5):\n",           o)
print("eye(4):\n",              e)
print("arange(0,20,2)    :", r)
print("linspace(0,1,6)   :", l)
print("random(3,3):\n",         R)

# --- manipulation ---
print("\n--- manipulation ---")
print("shape of A    :", A.shape)
print("reshape r→(2,5):\n", r.reshape(2, 5))
print("transpose A:\n",     A.T)
print("flatten A     :", A.flatten())
print("hstack a,a    :", np.hstack([a, a]))
print("vstack a,a:\n",      np.vstack([a, a]))
print("A + 10:\n",          A + 10)
print("A * 2:\n",           A * 2)
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 2 — Linear Algebra")
print("=" * 60)

A2 = np.array([[1, 2], [3, 4]])
B2 = np.array([[5, 6], [7, 8]])
v  = np.array([1, 2])

print("A:\n", A2)
print("B:\n", B2)

# dot product (vector)
print("\nDot product v·v      :", np.dot(v, v))

# matrix addition
print("Matrix addition A+B:\n", A2 + B2)

# matrix multiplication
print("Matrix multiply A@B:\n", A2 @ B2)
print("(same via np.dot):\n",   np.dot(A2, B2))
```

```
# scalar multiply, element-wise
print("Element-wise A*B:\n",     A2 * B2)

# determinant & inverse
print("det(A)                :", np.linalg.det(A2))
print("inv(A):\n",                np.linalg.inv(A2))
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 3 — Indexing & Slicing")
print("=" * 60)

M = np.arange(1, 26).reshape(5, 5)
print("M:\n", M)

print("\nM[0, :]       (row 0)       :", M[0, :])
print("M[:, 0]       (col 0)       :", M[:, 0])
print("M[1:4, 1:4]   (sub-matrix):\n", M[1:4, 1:4])
print("M[::2, ::2]   (every 2nd):\n",  M[::2, ::2])
print("M[[0,2,4], :] (fancy rows):\n", M[[0, 2, 4], :])

# boolean mask
mask = M > 15
print("Boolean mask M>15:\n",    mask)
print("Elements where M>15 :", M[mask])
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 4 — Row Operations")
print("=" * 60)

def row_ops_demo(mat):
    """Show the three elementary row operations on a copy."""
    M = mat.astype(float).copy()
    print("Original:\n", M)

    # swap rows 0 and 1
    M[[0, 1]] = M[[1, 0]]
    print("\nSwap rows 0↔1:\n", M)

    # scale row 1 by 3
    M[1] *= 3
    print("\nScale row 1 × 3:\n", M)

    # add 2× row 0 to row 2
    M[2] += 2 * M[0]
    print("\nRow2 = Row2 + 2×Row0:\n", M)
    return M

sample = np.array([[1, 2, 3],
                   [4, 5, 6],
                   [7, 8, 9]])
row_ops_demo(sample)
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 5 — LU Decomposition (scipy)")
print("=" * 60)

A5 = np.array([[2., 1., -1.],
               [-3., -1., 2.],
               [-2., 1., 2.]])
```

```
P, L, U = lu(A5)
print("A:\n", A5)
print("\nPermutation P:\n", P)
print("Lower L:\n", L)
print("Upper U:\n", U)
print("\nVerify P@L@U == A:", np.allclose(P @ L @ U, A5))
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 6 — Rotators (Givens rotation)")
print("=" * 60)

def givens_rotation(n, i, j, theta):
    """
    Return an n×n Givens rotation matrix G(i,j,theta).
    Rotating in the (i,j)-plane by angle theta.
    """
    G = np.eye(n)
    c, s = np.cos(theta), np.sin(theta)
    G[i, i] =  c;  G[i, j] = -s
    G[j, i] =  s;  G[j, j] =  c
    return G

n = 4
theta = np.pi / 4           # 45 degrees
G = givens_rotation(n, 0, 1, theta)
print(f"Givens G(0,1, 45°) [{n}×{n}]:\n", np.round(G, 6))

v6 = np.array([1., 0., 0., 0.])
print("\nv =", v6)
print("G @ v (rotated):", np.round(G @ v6, 6))
print("||G @ v|| (must = 1):", np.round(np.linalg.norm(G @ v6), 10))
print("G is orthogonal (G.T@G = I):", np.allclose(G.T @ G, np.eye(n)))
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 7 — Reflectors (Householder)")
print("=" * 60)

def householder(v):
    """
    Return the Householder reflector H = I - 2 v v^T / (v^T v)
    that reflects across the hyperplane orthogonal to v.
    """
    v = np.array(v, dtype=float).reshape(-1, 1)
    return np.eye(len(v)) - 2 * (v @ v.T) / (v.T @ v)

def householder_vec(x):
    """
    Build the Householder vector u s.t. H x = ±||x|| e1.
    Returns (H, u).
    """
    x = np.array(x, dtype=float)
    u = x.copy()
    u[0] += np.sign(x[0]) * np.linalg.norm(x)
    H = householder(u)
    return H, u

x7 = np.array([4., 3., 0.])
H7, u7 = householder_vec(x7)
```

```
print("x =", x7)
print("Householder vector u =", u7)
print("H:\n", np.round(H7, 6))
print("H @ x (should be ±||x||·e1):", np.round(H7 @ x7, 10))
print("H is symmetric  :", np.allclose(H7, H7.T))
print("H is orthogonal :", np.allclose(H7 @ H7.T, np.eye(len(x7))))
```

In [ ]:

```
print("\n" + "=" * 60)
print("TASK 8 — QR Decomposition via Givens Rotators")
print("=" * 60)

def qr_givens(A):
    """
    QR decomposition using Givens rotations.
    Returns Q, R  s.t. A = Q @ R, Q orthogonal, R upper-triangular.
    """
    m, n = A.shape
    Q = np.eye(m)
    R = A.astype(float).copy()

    for j in range(n):
        for i in range(m - 1, j, -1):
            # zero out R[i, j] using a Givens rotation in rows (i-1, i)
            a, b = R[i - 1, j], R[i, j]
            if b == 0:
                continue
            r = np.hypot(a, b)
            c, s = a / r, b / r

            # build and apply the full rotation
            G = np.eye(m)
            G[i - 1, i - 1] =  c;  G[i - 1, i] = s
            G[i,     i - 1] = -s;  G[i,     i] = c

            R = G @ R
            Q = Q @ G.T   # accumulate: Q = G1^T G2^T ...

        print(f"  after zeroing column {j}:")
        print("  R =\n", np.round(R, 6))

    return Q, R

A8 = np.array([[12.,  -51.,   4.],
               [ 6.,  167., -68.],
               [-4.,   24., -41.]])

print("A:\n", A8)
Q8, R8 = qr_givens(A8)
print("\nQ (Givens):\n",  np.round(Q8, 6))
print("R (Givens):\n",  np.round(R8, 6))
print("Q@R == A :", np.allclose(Q8 @ R8, A8))
print("Q orthogonal:", np.allclose(Q8.T @ Q8, np.eye(Q8.shape[0])))

# compare with numpy's built-in
Q_np, R_np = np.linalg.qr(A8)
print("\nnp.linalg.qr  Q:\n", np.round(Q_np, 6))
print("np.linalg.qr  R:\n", np.round(R_np, 6))
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 9 — QR Decomposition via Householder Reflectors")
print("=" * 60)

def qr_householder(A):
    """
    QR decomposition using Householder reflectors.
    Returns Q, R  s.t. A = Q @ R.
    """
    m, n = A.shape
    Q = np.eye(m)
    R = A.astype(float).copy()

    for k in range(min(m - 1, n)):
        x = R[k:, k]                    # sub-column to zero below diagon
        e1 = np.zeros_like(x); e1[0] = 1.0
        u = x + np.sign(x[0]) * np.linalg.norm(x) * e1
        if np.linalg.norm(u) < 1e-14:
            continue
        u = u / np.linalg.norm(u)

        # apply reflector to the trailing sub-matrix
        R[k:, k:] -= 2 * np.outer(u, u @ R[k:, k:])

        # accumulate Q
        Q[k:, :] -= 2 * np.outer(u, u @ Q[k:, :])

        print(f"  after reflector {k}:")
        print("  R =\n", np.round(R, 6))

    return Q.T, R

A9 = A8.copy()
print("A:\n", A9)
Q9, R9 = qr_householder(A9)
print("\nQ (Householder):\n", np.round(Q9, 6))
print("R (Householder):\n", np.round(R9, 6))
print("Q@R == A :", np.allclose(Q9 @ R9, A9))
print("Q orthogonal:", np.allclose(Q9.T @ Q9, np.eye(Q9.shape[0])))
```

In [ ]:
```
print("\n" + "=" * 60)
print("TASK 10 — Rectangular System via Row Operations vs lstsq")
print("=" * 60)

def gauss_elim_solve(A, b):
    """
    Solve A x = b for a full-rank square or tall rectangular system
    by Gaussian elimination (no pivoting for clarity).
    If A is rectangular (m>n), reduces to square and back-substitutes.
    """
    m, n = A.shape
    # build augmented matrix [A | b]
    Ab = np.hstack([A.astype(float), b.astype(float).reshape(-1, 1)])
    pivot_rows = []

    col = 0
    row = 0
    while row < m and col < n:
        # find pivot
```

```
        max_row = np.argmax(np.abs(Ab[row:, col])) + row
        if np.abs(Ab[max_row, col]) < 1e-12:
            col += 1
            continue
        Ab[[row, max_row]] = Ab[[max_row, row]]  # swap
        pivot_rows.append(col)
        Ab[row] /= Ab[row, col]                  # scale pivot to 1
        for r in range(m):
            if r != row:
                Ab[r] -= Ab[r, col] * Ab[row]    # eliminate
        row += 1; col += 1

    x = Ab[:n, -1]
    return x

# over-determined system: 5 equations, 3 unknowns
np.random.seed(42)
A10 = np.random.randint(1, 10, (5, 3)).astype(float)
x_true = np.array([1., 2., 3.])
b10 = A10 @ x_true + 0.01 * np.random.randn(5)   # slight noise

print("A (5×3):\n", A10)
print("b:", b10)

# --- row operations on normal equations A^T A x = A^T b ---
ATA  = A10.T @ A10
ATb  = A10.T @ b10
x_row = gauss_elim_solve(ATA, ATb)
print("\nx via row-ops (normal eqns):", np.round(x_row, 6))

# --- built-in least squares ---
x_lstsq, _, _, _ = np.linalg.lstsq(A10, b10, rcond=None)
print("x via np.linalg.lstsq      :", np.round(x_lstsq, 6))
print("Max difference             :", np.max(np.abs(x_row - x_lstsq)))
```

In [ ]:

```
print("\n" + "=" * 60)
print("TASK 11 — Rectangular System via QR Decomposition vs lstsq")
print("=" * 60)

def qr_solve(A, b):
    """
    Least-squares solution of A x = b using QR decomposition.
    For A (m×n) with m >= n full column rank:
        A = Q R  →  R x = Q^T b  (back-substitution)
    """
    Q, R = np.linalg.qr(A)
    Qt_b = Q.T @ b            # reduce to R x = Q^T b
    n = R.shape[1]
    R1   = R[:n, :n]          # square upper-triangular part
    rhs  = Qt_b[:n]
    # back-substitution
    x = np.zeros(n)
    for i in range(n - 1, -1, -1):
        x[i] = (rhs[i] - R1[i, i+1:] @ x[i+1:]) / R1[i, i]
    return x

x_qr    = qr_solve(A10, b10)
x_lstsq2, _, _, _ = np.linalg.lstsq(A10, b10, rcond=None)

print("A (5×3):\n", A10)
```

```
print("b:", b10)
print("\nx via QR solve             :", np.round(x_qr, 6))
print("x via np.linalg.lstsq      :", np.round(x_lstsq2, 6))
print("Max difference             :", np.max(np.abs(x_qr - x_lstsq2)))
```

In [ ]:

```
print("\n" + "=" * 60)
print("TASK 12 — Least Squares Full Pipeline")
print("=" * 60)

np.random.seed(7)
m_data = 20            # number of observations
deg    = 3             # polynomial degree  →  n = deg+1 unknowns

# ground truth coefficients [c0, c1, c2, c3]
c_true = np.array([1.0, -2.0, 0.5, 0.1])

# --- generate data ---
t = np.linspace(-3, 3, m_data)
y = np.polyval(c_true[::-1], t) + 0.3 * np.random.randn(m_data)

# --- build Vandermonde data matrix ---
# columns: 1, t, t^2, t^3
Phi = np.column_stack([t**k for k in range(deg + 1)])    # shape (20, 4)
print(f"Data matrix Phi shape: {Phi.shape}")
print("First 4 rows of Phi:\n", np.round(Phi[:4], 4))
print("y (first 5):", np.round(y[:5], 4))

# --- normal equations Phi^T Phi c = Phi^T y ---
PhiTPhi = Phi.T @ Phi
PhiTy   = Phi.T @ y
print("\nNormal equation matrix Phi^T Phi:\n", np.round(PhiTPhi, 4))
print("Right-hand side Phi^T y         :", np.round(PhiTy, 4))

# --- solve via row operations ---
c_row = gauss_elim_solve(PhiTPhi, PhiTy)
print("\nc via row-ops (normal eqns):", np.round(c_row, 6))

# --- solve via QR decomposition ---
c_qr = qr_solve(Phi, y)
print("c via QR decomposition      :", np.round(c_qr, 6))

# --- built-in least squares ---
c_lstsq, _, _, _ = np.linalg.lstsq(Phi, y, rcond=None)
print("c via np.linalg.lstsq       :", np.round(c_lstsq, 6))

# --- polyfit for extra comparison ---
c_poly = np.polyfit(t, y, deg)[::-1]   # polyfit gives highest-degree fir
print("c via np.polyfit            :", np.round(c_poly, 6))

print("\nTrue coefficients           :", c_true)
print("\nMax |row-ops - lstsq|:", np.max(np.abs(c_row   - c_lstsq)))
print("Max |QR     - lstsq|:", np.max(np.abs(c_qr    - c_lstsq)))
print("Max |polyfit - lstsq|:", np.max(np.abs(c_poly  - c_lstsq)))
```